



Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness



COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Flutter Web Client-Side Interface for Your Own Project

Flutter Web Client-Side Design

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 6
Output	Flutter UI-flow diagram, responsive wireframes/prototype, widget and validation plan, and design review
Maximum Score	80 points

Student / Project Information

Block: BSIT 3A SEMQ3	Date: September 19, 2026
System Title: RoadWatch - Pothole Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System	Instructor: Jean Y. Villasin, MIT
Members: Cy Monzon; Mae D. Pelipas; Katrice B. Anunciado (confirm official spelling and school roster)	

Draft design v0.1: group verification, instructor/peer review and responsive browser tests are pending.

1. Activity Alignment and Purpose

Topic Learning Outcome: Design the Flutter Web user interface, navigation flow, validation plan, and responsive behavior for the semester system.

Course Outcome Alignment: Supports semester-project client-side design by converting approved requirements and use cases into a usable, accessible, responsive, and implementation-ready Flutter Web interface plan.

Syllabus Assessment: Flutter UI flow/wireframe; design review; online interface-design exercise.

Purpose of the Activity

Activity 5 established the requirements baseline and system-specific use case diagram. In Activity 6, your team will translate those approved user goals and requirements into the actual client-side experience of your semester system. You will plan the screen sequence, navigation and routes, responsive layouts, reusable Flutter widgets/components, forms and validation behavior, accessibility and usability decisions, and the loading, empty, success, and error states that users will encounter. The final UI-flow and responsive wireframes will serve as the implementation blueprint for the Flutter Web client in the succeeding development activities.

Important distinction: A wireframe defines structure, hierarchy, content placement, and interaction flow; a prototype simulates interaction; the final Flutter interface is the implemented product. Client-side validation improves usability but does not replace validation and authorization in the REST API.

2. Intended Learning Objectives

- Translate the approved SRS, system scope, user roles, and use cases into a coherent Flutter Web screen flow.
- Identify appropriate widget-based UI structures and distinguish presentational/static components from components that require local or shared state.
- Plan navigation, named routes or route patterns, protected screens, back/exit behavior, and user-role access to major screens.
- Create responsive wireframes that adapt intentionally across representative mobile, tablet, and desktop web widths.
- Design reusable interface components and a consistent visual system for forms, cards, navigation, lists, dialogs, feedback, and common actions.
- Define client-side form validation and loading, empty, success, and error states for important user workflows.
- Review the design for accessibility, usability, requirement traceability, and implementation readiness before Flutter coding begins.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; every member must contribute to flow analysis, wireframing, review, and oral explanation.
Suggested Time	Approximately 3 laboratory hours plus the scheduled design review and online interface-design exercise.
Primary Output	One system-specific Flutter UI-flow diagram and a responsive wireframe/prototype set based on the approved project requirements.
Supporting Output	Screen inventory; widget/state plan; route map; reusable-component plan; form-validation plan; UI-state matrix; accessibility/usability checklist; design-review revision log.
Recommended Tools	Figma, Penpot, diagrams.net/Draw.io, Canva, PowerPoint, Flutter/Dart for optional clickable prototype work, or an equivalent instructor-approved design tool.
Design Review	5–7 minutes per group followed by 2–3 minutes of questions. Every member must be able to explain assigned screens, routes, responsive decisions, and interface states.

4. Pre-Laboratory Requirements

- Approved or revised SRS, user roles/permissions, scope, requirements-to-feature matrix, and system-specific UML use case diagram from Laboratory Activity No. 5.
- Prescribed Flutter Web-REST API-PostgreSQL technology-stack blueprint from Laboratory Activity No. 3 and current architecture diagram from Laboratory Activity No. 2.
- Selected development methodology and semester milestones from Laboratory Activity No. 4.
- List of Must/High-priority features and the user workflows that must be demonstrated during the semester project.
- Any existing branding, content, icons, forms, data samples, or interface references relevant to the team's actual system.
- A shared design workspace or project repository where the UI-flow, wireframes, screenshots, and review notes can be versioned.

5. Part A – Translate Requirements and Use Cases into the UI Flow

Start with the approved requirements and use case diagram—not with visual styling. For each important user goal, identify where the user enters the interface, which screens or overlays are required, where decisions or validation occur, and what successful completion looks like.

Use Case / Req. ID	Actor / Role	Entry Point	Required Screen Sequence	Decision / Validation Point	Expected Destination
UC-06; FR-11/14	Monitoring User	Open dashboard	Overview -> Detection Records	API request succeeds; available data or empty/error state	Detection Records: potholes only
UC-06; FR-05/11	Monitoring User	Overview / navigation	Overview -> Pothole Map	GPS exists? If absent, show no-location/no- data message	Pothole Map: valid GPS records only
UC-06; FR-11/14	Monitoring User	Detection Records	Search or sort -> review list	Query may return zero; failed API fetch shows retry	Filtered pothole list or recoverable error
FR-02/03; UC-02	Mobile User / Driver	Mobile monitoring (separate client)	Camera -> on-device model -> pothole-only alert/save	Class 3 only; classes 0- 2 ignored	Pothole-only data eligible for web display
FR-07/08; UC-05/07	Mobile User / Driver	Offline detection (separate client)	Queued record -> connectivity -> REST API	If API unavailable keep queued; no web record until sync	Pothole record available on web after successful sync

Traceability Rule

Every Must/High-priority user-facing requirement should lead to one or more identifiable screens, actions, or interface states. If the wireframes show a major feature that is not supported by the approved requirements, flag it for requirements review before implementation.

Design basis: Activity 5 draft requirements FR-01 to FR-14, NFR-01 to NFR-08 and UC-01 to UC-07; six dashboard screenshots; Agile/Scrum; and observed navigation paths. Authentication and administrator editing remain unverified. Android detection is a separate client; the web dashboard technology still requires confirmation.

The web flow follows the existing dashboard. Android detection/queuing appears only as a separate data-source boundary, not as an invented web screen.

RoadWatch UI-flow diagram - Part A

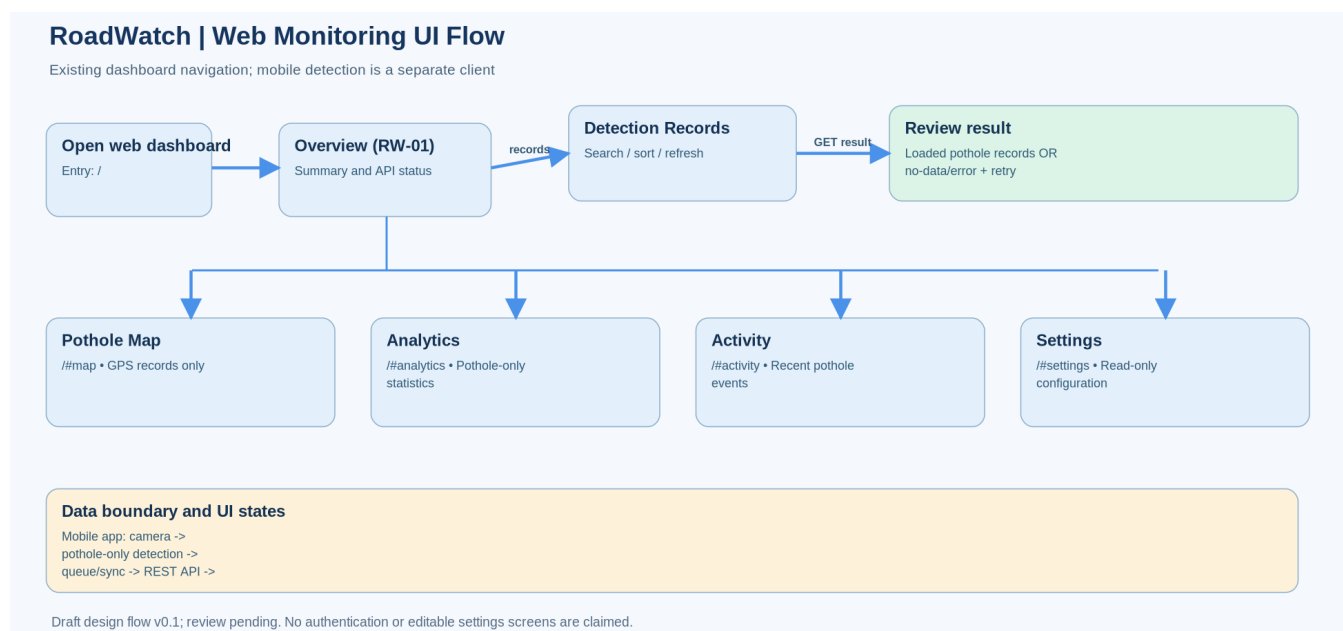


Figure 1. Web UI flow with existing routes and separate mobile data source. Editable source: roadwatch-ui-flow.drawio.

6. Part B – Screen Inventory and Widget-Based UI Planning

Create a complete screen inventory before polishing individual wireframes. Use actual project screen names. The “widget/state plan” is a design annotation—not a requirement to write the final Dart code during this activity.

Screen ID / Name	Purpose / Main User	Major UI Sections / Candidate Flutter Widgets	Reusable Component Candidates	State Needed	Data / API Dependency
RW-01 Overview	Summary cards, status, detection summary; Monitoring User	Scaffold Id, AppBar, Drawer, Cards, status chip	Header, metric card, feedback	API fetch/error, refresh	GET summary/status via REST API
RW-02 Pothole Map	GPS pothole markers or no-location message; Monitoring User	Map container, detection-type dropdown, legend	Header, status, empty- state	Map loading, marker selection	Map tiles and geolocated records via REST API
RW-03 Detection Records	Review/search/sort pothole records; Monitoring User	Search field Id, fixed Potholes filter, sort, refresh, table/card	Header, record row, status, buttons	Query, sort, loading/errors	GET pothole records; search/sort fetched data
RW-04 Analytics	Pothole statistics and confidence summary; Monitoring User	Cards, chart panels, feedback banner	Header, metric card, chart panel	Async summary/chart data	GET pothole aggregates/statistics
RW-05 Activity	Recent pothole events and activity timeline; Monitoring User	Summary cards, recent-event list, refresh	Header, event row, feedback	Async event data	GET recent pothole events
RW-06 Settings	Show read-only model/API/database configuration; Monitoring User	Static information cards, config rows, read-only notice	Header, info card, label/value	API availability indicator (not editable)	Read-only model/API/database configuration

Widget/state plan: labels, info cards and read-only settings can be presentational widgets. Search, sort, active menu and map selection use local interaction state; fetched records and API request states need shared/asynchronous state. These are plans, not claims that current dashboard is coded in Flutter Web.

Widget and State Classification Guide

Design Question	Likely UI Planning Direction	Examples / Notes
Does the component mainly render information from inputs and have no local mutable interaction state?	Plan it as a presentational/reusable widget; it may later be implemented as a StatelessWidget or equivalent composition.	Header, label, summary card, icon-text row, static empty-state panel.
Does the component own temporary local UI state that changes during interaction?	Identify the local state and event that changes it; a StatefulWidget may be appropriate during implementation.	Password visibility, tab selection, expansion, local form interaction.
Is the state shared across screens or tied to asynchronous application data?	Do not force it into one StatefulWidget. Identify the shared/application-state boundary and planned state-management approach.	Authenticated user, loaded records, filters used across screens, async request state.
Can the same structure or behavior appear in several screens?	Extract it as a reusable component with clearly defined inputs, callbacks, and states.	Buttons, form fields, cards, status chips, confirmation dialogs, table/list rows.

Possible Flutter building blocks to annotate when appropriate: Scaffold, AppBar, NavigationRail/Drawer, Row, Column, Wrap, GridView, ListView, Card, Form, TextFormField, DropdownButton/FormField, buttons, dialogs, LayoutBuilder, MediaQuery, and project-specific reusable widgets.

7. Part C – Navigation and Routing Plan

Plan how users move between screens and what should happen when they use browser navigation, direct URLs, sign in/out, role-restricted routes, or invalid routes. The final route structure must match the approved user roles and scope.

Route / Path	Screen / View	Allowed Role(s)	Entered From	Primary Exit / Back Behavior	Guard / Condition
/	Overview entry / existing index	Monitoring User	Open dashboard	Overview; browser back to previous site	No sign-in screen shown in provided evidence
/#overview	Overview	Monitoring User	Sidebar or initial entry	Stay within dashboard	GET failure shows truthful offline state
/#map	Pothole Map	Monitoring User	Sidebar/overview	Browser back -> prior route	Display only records with GPS
/#detections	Detection Records	Monitoring User	Sidebar/overview	Browser back -> prior route	Read-only review; no edit/delete action
/#analytics	Analytics	Monitoring User	Sidebar	Browser back -> prior route	Pothole-only summary data
/#activity	Activity	Monitoring User	Sidebar	Browser back -> prior route	Pothole-only recent events
/#settings	Settings (read-only)	Monitoring User; Admin (view only)	Sidebar	Browser back -> prior route	No settings modification UI
Unknown hash	Fallback to Overview (planned)	Any dashboard viewer	Invalid URL	Show friendly message; return to Overview	Do not imply authentication exists

Observed hash paths are preserved. The project screenshots do not show a login screen or permissions enforcement; do not invent protected routes. If instructor requires authentication, revise the SRS first.

Critical path: Open / Overview Detection Records choose Potholes/search/sort API loading pothole list OR no → → → → → results OR recoverable fetch error Refresh. Browser back returns to previous hash view; unknown route returns to → Overview with a message (proposed).

- Show the normal route sequence for at least one end-to-end critical workflow.
- Mark routes that require authentication, a specific role, ownership of a record, or another access condition.
- Define what the interface should do for an unknown/invalid route and after session expiration.
- Design navigation so that browser back/forward behavior and direct URLs do not create confusing or unauthorized states.

8. Part D – Responsive Layout and Wireframe Planning

Flutter Web must respond to available space rather than merely shrink a desktop canvas. Use representative widths to test the layout. The exact breakpoints may differ by project, but the team must demonstrate intentional adaptation.

Screen / Module	Narrow / Mobile Behavior	Medium / Tablet Behavior	Wide / Desktop Behavior	Overflow / Long-Content Strategy
Overview	Compact header/menu; metric cards in 1 column	Compact nav; 2-column cards	Full sidebar; 4 metric cards across	Wrap labels; vertical scroll

Screen / Module	Narrow / Mobile Behavior	Medium / Tablet Behavior	Wide / Desktop Behavior	Overflow / Long-Content Strategy
Pothole Map	Map full width; filters above; text empty state	Map with compact nav, full width	Sidebar + wide map and legend	Map pans; details stack below
Detection Records	Search full width; filter/sort next line; record cards	Search + grouped filters; records adapt	Sidebar + filter row + table/list	Wrap record fields; paginate/scroll vertically
Analytics	Metric cards stack; charts become single column	Two-column metric cards	Four summary cards and side-by-side panels	Chart labels wrap; no horizontal page overflow
Activity	Event cards stack vertically	Two-column summary as space permits	Three summary cards + timeline	Event list scrolls vertically
Settings	One-column read-only info rows	Two-column information cards	Wide grid of configuration information	Long values wrap; no edit controls

Minimum Responsive Wireframe Requirement

Produce wireframes for all major screens at the team's primary desktop/web width. In addition, show at least three critical screens at three representative widths (for example, approximately 390 px, 768 px, and 1440 px) so the reviewer can see how navigation, columns, cards, forms, tables/lists, spacing, and actions adapt. Do not rely on tiny text or uncontrolled horizontal overflow as the responsive solution.

Required responsive examples: RW-01 Overview, RW-02 Pothole Map and RW-03 Detection Records each at 390px, 768px and 1440px. Desktop wireframes for RW-04, RW-05 and RW-06 are also included. These show intended adaptation, not completed browser testing.

9. Part E – Reusable Components and Visual Consistency Plan

Identify the recurring pieces of the interface before implementation. Reusable components should reduce duplicated UI logic while preserving consistent behavior and visual language across the system.

Component / Pattern	Purpose	Used in Screens	Inputs / Variants	Interactive State / Callback	Accessibility / Usability Note
Primary action button	Refresh display	Overview, Records, Activity	enabled/loading/disabled	busy, click -> GET request	Visible text and keyboard focus
Search input	Narrow record list	Detection Records	query string	local input/filter	Visible label; clear empty query
Navigation item	Switch web screens	All six	active/inactive	click -> hash route	Focus indicator and active text
Record row/card	Review pothole metadata	Detection Records, Activity	confdience, GPS, time when available	optional selection	Readable labels; no crack entries
Status/feedback banner	Report API success/failure	Overview, Records, Analytics, Activity	loading/empty/success/error	retry available if safe	Text + icon, not color alone
Configuration info row	Display read-only settings	Settings	key/value	no change callback	Mark clearly read-only

Create a simple interface consistency guide covering: typography hierarchy, spacing system, primary/secondary actions, form-field states, color usage, cards/containers, icon conventions, navigation patterns, confirmation behavior, and feedback messages.

10. Part F – Forms and Input Validation Plan

For each important form, define both the visible interaction and the validation behavior. The message shown to the user should explain what must be corrected. Client-side checks are for immediate feedback; the REST API must still validate submitted data and enforce permissions.

Visual consistency: retain existing dark navy theme; use heading/body hierarchy, 8/16/24px spacing, one clear primary action per area, visible labels and text-based status. Use the same sidebar/compact navigation, feedback banners and no-data wording on every screen. Do not rely on red/green status colors alone.

pi RoadWatch dashboard is a read-only reviewer interface. There is NO demonstrated record-entry form or editable Settings form. This table covers existing search, filter, sort, refresh and GPS display controls rather than fabricating a submission form.

Form / Field	Input / Control	Client-Side Rule	When Error Appears	Error / Helper Message	Submit / API Behavior
Records search	Text search	Trim whitespace; blank shows all	After typing or submit	No matches for this search.	Client-side filter or GET; no record mutation
Records detection type	Fixed Potholes value	Do not permit crack values	When selected	Only potholes are monitored.	GET pothole records only
Records sort	Newest first / other existing sort options	Use valid sort choice	On selection	Choose an available sort order.	Reorder fetched records; not a submission

Form / Field	Input / Control	Client-Side Rule	When Error Appears	Error / Helper Message	Submit / API Behavior
Map GPS filter	Pothole-only map selection	Map only valid coordinates when present	On data load	No pothole records with GPS yet.	Read-only GET; never invent missing GPS
Refresh	Button	Disable repeated click while pending	During request	Unable to load. Check connection and retry.	GET only; keep currently displayed data if possible
Settings (read-only)	Information only	No editable fields in current UI	Not applicable	Configuration is read-only.	No settings update API call

- Consider required fields, length, format, numeric/date range, cross-field rules, file type/size, duplicate input, and conditional fields when relevant.
- Show disabled/submitting states so users cannot accidentally submit the same action repeatedly.
- Decide where validation is inline, on field exit, on submit, or a combination—then apply the pattern consistently.

11. Part G – Accessibility and Usability Review

Review the wireframes from the perspective of a user who may navigate by keyboard, use assistive technology, have reduced vision, or encounter the system under time pressure or unreliable connectivity.

Review Area	Required Design Evidence	Status / Revision Note
Labels and instructions	Controls and inputs have visible, understandable labels; placeholder text is not the only label.	Search and dropdown have visible labels; cards label quantities Planned - check during review
Keyboard and focus	Interactive controls have a logical focus order and visible focus state; key workflows do not require a mouse.	Sidebar, refresh, search, select support visible keyboard focus Planned - test in browser
Contrast and non-color cues	Text and controls remain understandable with sufficient contrast; status/error meaning is not conveyed by color alone.	Dark theme text contrast; pair red/green status with text/icon Planned - test contrast
Text and layout resilience	Important content remains readable when text is enlarged or the viewport becomes narrow.	No tiny 3-column cards at 390 px; stack elements and wrap labels Shown in wireframes; test in Flutter Web
Touch/click targets	Buttons, icons, checkboxes, and menu items are large enough and spaced to prevent accidental activation.	Use comfortably sized nav, filter and refresh controls Planned - test at mobile width
Feedback and recovery	Errors identify the problem and recovery action; destructive actions use suitable confirmation/undo patterns.	Show data-load error, preserved filters and refresh/retry Shown in records wireframe
Semantics / meaningful content	Images/icons and page structure have meaningful labels/semantics where they communicate information.	Map and chart get textual summary/empty message Planned - implement accessible labels
Consistency and cognitive load	Navigation, actions, terminology, and form behavior stay consistent across screens.	Same sidebar/compact menu, labels and feedback across six screens Shown in wireframes

12. Part H – Loading, Empty, Success, and Error-State Design

Do not design only the ideal “happy path.” Every major data-driven screen or submit action should have a planned state for waiting, no data, completion, and failure.

Screen / Action	Loading State	Empty / No-Data State	Success State	Error / Recovery State	Access / Offline Note
Overview	Loading cards/indicator	No pothole records yet	Summary counts & health data	API unavailable; retry/refresh	Status text must match real fetch state
Pothole Map	Map loading indicator	No geolocated pothole records	Markers for valid pothole GPS	Map/data fails -> retry	GPS missing does not create a pin
Detection Records	Loading rows	No potholes match current search	Rows/cards with actual API records	Failed to fetch -> retry	Search/sort local until data loads
Analytics	Loading chart placeholder	No records to summarize	Pothole statistics shown	Unable to load analysis -> retry	Never show crack classes

Screen / Action	Loading State	Empty / No-Data State	Success State	Error / Recovery State	Access / Offline Note
Activity	Loading event list	No recent pothole activity	Recent pothole events displayed	Unable to load activity -> retry	Events refel ct successful API records
Settings	Reading configuration	Unavailable setting noted	Read-only configuration visible	API status uncertain -> truthful warning	Viewing does not grant edit rights

State-Design Rule

Each state should answer four questions: What happened? What is the system doing now? What can the user do next? Is the action safe to retry? Loading indicators should not hide indefinitely, empty states should offer a meaningful next step when appropriate, and error states should preserve user work whenever possible.

Actual screenshot evidence: Detection Records, Analytics and Activity currently show failed fetch, while the header sometimes says API Connected. This is a UX mismatch to review, not evidence that the API or database is functioning. A failed fetch must not be displayed as a genuine zero-count result.

13. Part I – Construct the Flutter UI Flow and Responsive Wireframes

Create one coherent design package that a teammate could use to begin Flutter implementation without guessing the basic navigation, screen hierarchy, responsive behavior, validation, and interface states.

No.	Required Element	What Must Be Shown
1	System identity	Official system title, group, design version/date, and the user role(s) represented in the flow.
2	Screen inventory	Every major user-facing screen required by the approved project scope, using stable screen IDs/names.
3	UI-flow diagram	Directional links between screens/overlays, with labels for important user actions, decisions, and success/error branches.
4	Navigation and routing	Planned route/path or route name, main navigation structure, role restrictions, and important entry/exit behavior.
5	Widget-based structure	Annotated sections/components showing suitable Flutter UI building blocks and the separation of reusable components.
6	State planning	Identify where local interaction state, shared/application state, or asynchronous request state is required.
7	Responsive behavior	Desktop/wide layout plus responsive adaptations; at least three critical screens demonstrated at three representative widths.
8	Forms and validation	Key inputs, controls, validation conditions, error feedback, submit behavior, and disabled/loading states.
9	Accessibility/usability	Labels, focus/keyboard behavior, readable hierarchy, contrast/non-color cues, target sizing, and recovery guidance.
10	System feedback states	Loading, empty, success, and error/retry states for important data screens and transactions.
11	Requirement traceability	Wireframe screens/actions are labeled or cross-referenced to the relevant use cases, requirements, or Must/High-priority features.
12	Design consistency	Common typography, spacing, component behavior, terminology, and action placement are applied consistently across the design.

Minimum UI-Flow Rule

The design package must contain at least one complete critical path from user entry or sign-in -> navigation -> feature screen -> form/detail action -> validation/request state -> success or recoverable error -> updated destination. The flow must be understandable even without opening the Flutter code.

RoadWatch design evidence - Part I

No.	Required Element	RoadWatch design evidence
1	System identity	RoadWatch - Pothole Detection Using Mobile Dashcam Imagery with a Web- Based Monitoring System; design v0.1; Monitoring User.
2	Screen inventory	RW-01 through RW-06, matching existing sidebar.
3	UI-flow diagram	roadwatch-ui-flow.png and .drawio (also SVG).
4	Navigation and routing	Part C hash paths and invalid-route proposal.
5	Widget-based structure	Part B components and state boundaries.
6	State planning	Part B local and asynchronous distinctions; Part H.
7	Responsive behavior	Nine 390/768/1440px variants across RW-01..RW-03.
8	Forms and validation	Part F; no record-creation form is invented.
9	Accessibility/usability	Part G; review pending.

No.	Required Element	RoadWatch design evidence
10	System feedback states	Part H; distinguish empty from fetch failure.
11	Requirement traceability	Part A links FR/UC to web screens and mobile data boundary.
12	Design consistency	Part E and all wireframes follow existing dark UI.

See the accompanying 12 annotated wireframes (all six major desktop screens; Overview, Map and Records also in mobile/tablet variants), Figure 1 UI flow, editable SVG sources and editable diagrams.net file. No changes to the application code are required by this design activity.

Responsive wireframe coverage - Part I

Screen	390px mobile	768px tablet	1440px desktop
Overview	Included in source package	Included in source package	Included in source package
Pothole Map	Included in source package	Included in source package	Included in source package
Detection Records	Included in source package	Included in source package	Included in source package
Analytics	Not required	Not required	Included in source package
Activity	Not required	Not required	Included in source package
Settings	Not required	Not required	Included in source package

The supplied RoadWatch PDF contains the following three desktop examples. Its text refers to a separate 12-variant wireframe package and editable sources; those separate files are not embedded in this worksheet PDF.

Part I - Overview desktop wireframe

RW RoadWatch

Pothole Monitoring

Overview

Pothole Map

Detection Records

Analytics

Activity

Settings

Overview

RoadWatch | Pothole monitoring

Monitoring status

API data availability must match latest fetch result

TOTAL DETECTIONS

-- (pending API data)

POTHOLE DETECTIONS

-- (pending API data)

AVG. CONFIDENCE

-- (pending API data)

LATEST DETECTION

-- (pending API data)

Pothole Detection Summary

Only Pothole class is shown

No records yet / failed fetch shown as separate states

System health

FastAPI and PostgreSQL status shown accurately

Status: API request pending

DESIGN WIREFRAME | 1440px | Overview | v0.1

Design based on existing RoadWatch UI; not an implementation screenshot.

Part I - Pothole Map desktop wireframe

RW RoadWatch

Pothole Monitoring

Overview

Pothole Map

Detection Records

Analytics

Activity

Settings

Pothole Map

RoadWatch | Pothole monitoring

Detection type: Potholes

Fixed production target; no crack filters

No pothole detections available

Markers appear only when GPS coordinates exist

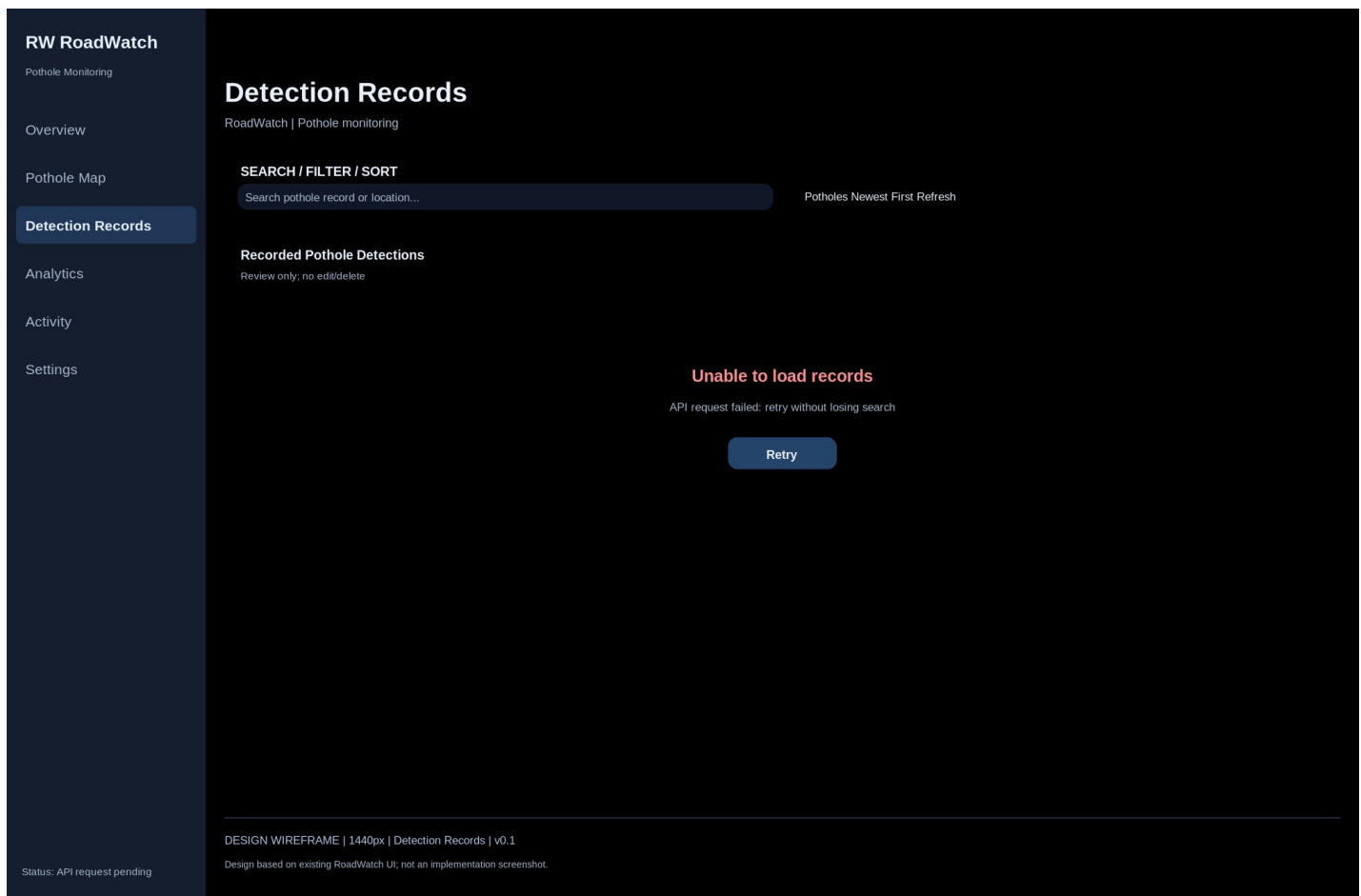
Legend

- Pothole | GPS-available records only

DESIGN WIREFRAME | 1440px | Pothole Map | v0.1

Design based on existing RoadWatch UI; not an implementation screenshot.

Status: API request pending

Part I - Detection Records desktop wireframe

Full-resolution PNG and editable SVG files are provided inside the wireframes/ folder. Three-width variants: 390px, 768px and 1440px. The example images below show the major web layouts; the files contain all 12 variants.

14. Reference Example – Do Not Copy as Your Final Output

The example below shows the level of flow clarity expected. Replace every label, screen, state, action, and route with the actual design of your approved semester system.

Reference only: derive the actual screens, routes, states, and responsive behavior from your own SRS and use cases.

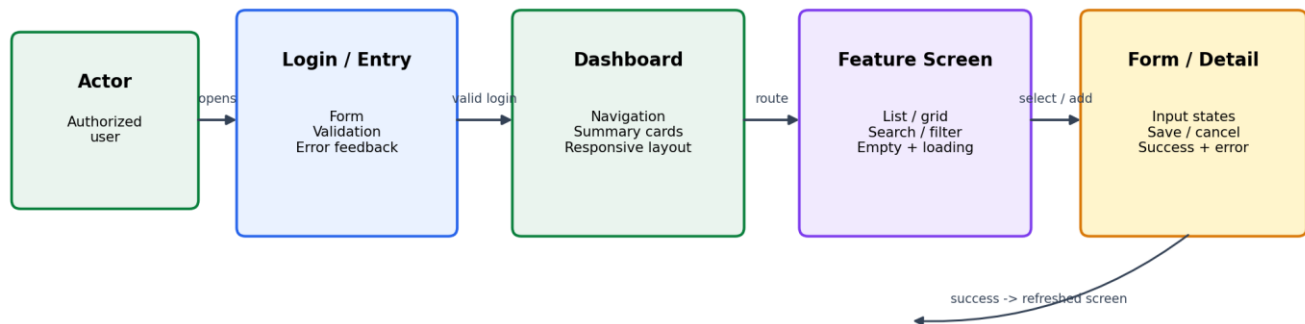


Figure 1. Reference client-side flow showing screens, navigation, validation, and interface states.

15. Recommended Flutter UI Design Annotations

Area	Possible Annotation	What It Communicates
Page shell	Scaffold / responsive shell	Overall page structure, navigation, content area, and global feedback region.
Navigation	NavigationRail / Drawer / menu / tabs	How primary destinations change by viewport and role.
Layout	Row / Column / Wrap / Grid / List	How content groups, wraps, stacks, scrolls, or changes column count.
Form	Form / TextFormField / dropdown / date input	Input grouping, validation feedback, submit behavior, and field states.
Reusable UI	ProjectCard / StatusChip / AppButton / FieldLabel	A project-specific reusable component and its variants.
Responsive logic	LayoutBuilder / MediaQuery / width class	Where structure or component presentation changes because of available space.
Async state	loading / empty / success / error	What the user sees before, during, and after a data request or mutation.

Scaffold/AppBar + responsive Drawer/NavigationRail; Row/GridView/Column; reusable cards and status banners; LayoutBuilder width classes; local search/sort and async API loading/empty/success/error states. These are candidate widgets for a Flutter Web implementation, not a representation of the present web dashboard source.

16. Laboratory Procedure

Phase 1 – Review the approved requirements (20 minutes)

Identify the Must/High-priority use cases, user roles, permissions, and major data interactions that require client-side screens.

Phase 2 – Build the screen inventory and UI flow (25 minutes)

List the screens, assign stable screen IDs, connect them using the actual user workflow, and identify decisions, guards, and important feedback states.

Phase 3 – Plan navigation, widgets, and reusable components (30 minutes)

Define the route structure, primary navigation pattern, major widget groups, reusable components, and state boundaries.

Phase 4 – Draft responsive wireframes (50 minutes)

Create the major desktop/web wireframes and the required responsive variants for at least three critical screens.

Phase 5 – Add validation and interface states (20 minutes)

Annotate forms, client-side validation, loading, empty, success, and error/recovery behavior.

Phase 6 – Accessibility and usability review (15 minutes)

Check keyboard/focus behavior, labels, contrast/non-color cues, layout resilience, target sizing, and user recovery.

Phase 7 – Conduct peer/instructor design review (15 minutes)

Use the review log below. Record defects, unclear flows, missing states, requirement mismatches, and responsive problems.

Phase 8 – Revise, export, and submit (5 minutes)

Update the design version, export required files, commit the design artifacts, and submit the required links/files through google classroom.

17. Part J – Design Review and Revision Log

Use this log during the group, peer, or instructor design review. Do not silently fix major findings; record the problem and the revision decision so the team can explain how the design improved.

Issue ID	Reviewer Finding / Question	Affected Screen / Use Case	Priority	Revision Decision / Action	Status
DR-01	Screenshots show API Connected while record views say Failed to fetch	Overview/Records/ Analytics/Activity	High	Design one truthful status label tied to latest data request	Open: pending implementation and review
DR-02	Requirements mention administrator, but screenshot has no admin action	Settings	Medium	Keep read-only; do not add admin editing without approved SRS change	Open: confirm with group/instructor
DR-03	No genuine review/peer sign-off yet	All screens	Medium	Review mobile/tablet variants and record real findings	Pending peer/instructor review

Do not mark the design review as completed until a group member or instructor actually reviews it; replace these draft findings with documented review outcomes.

Phase 1-6: reviewed provided project scope, prepared screen inventory/routes, drafted responsive wireframes and validation/state plans, then listed accessibility checks. Phase 7-8: group and instructor design review, revisions, and LMS/repository submission remain pending.

18. Final Quality Checklist

Checklist A	Checklist B
☐ UI flow is derived from the approved SRS and use case diagram.	☐ Major screens are wireframed for the primary desktop/web width.
☐ Every Must/High-priority user-facing workflow maps to identifiable screens/actions.	☐ At least three critical screens show intentional mobile, tablet, and desktop adaptations.
☐ Screen names/IDs and route labels are consistent across the flow and wireframes.	☐ Layouts avoid unreadably small text and uncontrolled horizontal overflow.
☐ Protected or role-specific screens identify the relevant access condition.	☐ Keyboard/focus, labels, contrast/non-color cues, target size, and error recovery are reviewed.
☐ Reusable components are identified instead of duplicating the same pattern across screens.	☐ Terminology, buttons, spacing, navigation, and feedback patterns are consistent.
☐ Forms show required fields, validation behavior, error feedback, and submit/loading state.	☐ The design-review log records findings and the team's revision decisions.
☐ Loading, empty, success, and error/retry states are designed for important data workflows.	☐ Every member can explain assigned screens, routes, state behavior, and responsive choices without reading a script.

19. Submission Requirements

- Final Flutter UI-flow diagram in PDF or PNG format plus the editable source file.
- Responsive wireframe/prototype package showing all major screens and the required three-width variants for at least three critical screens; submit the editable file or accessible share link when the tool is web-based.
- Completed screen/widget inventory, route plan, reusable-component plan, validation plan, UI-state matrix, accessibility/usability review, and design-review revision log.
- Updated project repository or shared workspace under a folder such as docs/ui-design/ containing the current design version and exported review artifacts.
- Repository/share link and required files submitted through MasaoLMS using the naming convention IT110_LA6_SystemName.
- Complete the online interface-design exercise and incorporate instructor feedback that affects the project interface before implementation.

Academic Integrity and Competency Validation

The UI flow and wireframes must represent the team's own approved system, users, requirements, and design decisions. AI or design templates may be used to clarify concepts or accelerate drafting, but the submitted screen flow, labels, responsive choices, validation behavior, accessibility decisions, and state designs must be reviewed and understood by the team. The instructor may ask any member to trace a workflow or justify a screen/component decision without assistance.

RoadWatch checklist status: screen/route, component, validation and state plans are drafted. SRS approval, access restrictions, complete wireframe-package verification, accessibility/device testing, peer/instructor review and member oral readiness still require group confirmation.

20. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent with the approved requirements, technically weak, visually unclear, or not responsive. Design-review points may be reduced for unequal participation, excessive reading, or inability to explain the team's own interface decisions.

Criterion	Evidence for Full Credit	Points
Requirements-to-UI-flow traceability	The UI flow is complete for the approved scope; Must/High-priority use cases and requirements map clearly to screens, actions, decisions, and expected destinations.	10
Navigation and routing plan	Routes, navigation hierarchy, entry/exit behavior, role restrictions, protected screens, and important browser/back behavior are clear and internally consistent.	10
Responsive wireframes and layout design	Major screens are complete and readable; at least three critical screens demonstrate intentional adaptation across representative mobile, tablet, and desktop widths without uncontrolled overflow.	15
Widget structure and reusable components	The design shows sensible widget-based composition, identifies reusable UI patterns, and distinguishes local interaction state from broader/shared or asynchronous application state.	10
Forms, validation, and interface states	Important forms include usable validation and feedback; loading, empty, success, error/retry, disabled, and submit states are appropriately designed.	10
Accessibility and usability	The design addresses labels, focus/keyboard access, readable hierarchy, contrast/non-color cues, target sizing, consistency, user guidance, and recovery from errors.	8
Visual consistency and design documentation	Screen IDs, routes, terminology, typography, spacing, controls, annotations, and versioned design artifacts are clear, consistent, and implementation-ready.	7
Design review and oral defense	The group uses the UI flow/wireframes effectively, every member participates, revisions are documented, and the team can justify responsive, navigation, component, validation, and state decisions.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

21. Post-Laboratory Reflection

Which user workflow was the most difficult to translate from the SRS/use case diagram into a clear screen flow, and why?

Draft reflection (confirm with group): Detection Records: its search/sort steps need distinct loading, no-results and fetch-error outcomes. Our draft separates these states; group review is pending.

Which interface element should become a reusable Flutter widget/component first, and which screens will use it?

Draft reflection (confirm with group): A RecordStatus/Feedback component, used on Overview, Detection Records, Analytics and Activity for accurate loading/empty/error messages.

What was the most important change your team made between narrow/mobile, medium/tablet, and wide/desktop layouts?

Draft reflection (confirm with group): At narrow width, the wide sidebar becomes a compact menu and multi-column summary cards stack, while search and sort controls wrap.

Which validation, loading, empty, success, or error state would create the greatest usability problem if it were not designed before implementation?

Draft reflection (confirm with group): A failed API fetch must be shown as an error with retry, not confused with zero potholes.

What specific design issue must your team resolve before beginning the next Flutter client-side implementation activity?

Draft reflection (confirm with group): Confirm current dashboard technology and actual role restrictions, then verify API status messages and test the 390/768/1440px designs.